

News Recommender: Build Report

Datasets: MIND (English, MSN News) and EB-NeRD (Danish, Ekstra Bladet)

Repo: <https://github.com/TECHIE-TITAN/IRE-Assignment-News-Recommender>

Contents

1. What I built	1
2. Choices and why	2
3. Observations	3
3.1 Recall@K and re-ranking disagree about which retriever is better	3
3.2 The re-ranker works on MIND and actively hurts on EB-NeRD	3
3.3 The re-ranker trades tail performance for head performance	4
3.4 Bugs that changed results	4
4. Where it breaks at 10×	5
5. What I would do next	6

1. What I built

A two-stage news recommender that runs end to end on both datasets from the same code path. Stage one generates and scores candidates with lexical and semantic retrieval. Stage two re-ranks those candidates with a learned model over behavioural features.

Data layer. `pipeline/schema.py` defines one column contract for articles, interactions and click history. `pipeline/mind.py` and `pipeline/ebnerd.py` map two very different raw formats onto it. MIND ships TSV files with history embedded inline in each impression row. EB-NeRD ships parquet with history in a separate file that has to be joined on user id, and it carries real per-click timestamps instead of making you reconstruct them from positive labels. Everything downstream of the loaders is dataset agnostic, so adding EB-NeRD after MIND was mostly a loader plus a `--dataset` path prefix.

The split is temporal and comes from the dataset’s own file boundaries rather than a re-derived day cutoff. MIND trains on impressions up to 2019-11-14 23:59:59 and validates on 2019-11-15. EB-NeRD trains up to 2023-05-25 07:00 and validates on the following week. `pipeline/split.py` asserts the boundary is monotonic and that no click used to build a feature postdates the impression it feeds. Both datasets report `leakage_check: passed`, and there are two test files (`tests/test_leakage.py`, `tests/test_feature_leakage.py`) that assert the same property independently.

	MIND	EB-NeRD
Articles	104,151	20,738
Interactions	4,979,946	477,534
Click history rows	3,958,501	3,277,060
Train impressions	2,232,748	232,887
Val impressions	376,471	244,647
Test impressions (unlabelled)	2,370,727	13.5M
Mean candidates per impression	~37	~11.5
Mean history length	~30-40	~292

Retrieval. `retrieval/bm25.py` is a hand written BM25F over a sparse term-document matrix. Title and abstract are length normalised separately against their own average field length and then combined by field weight before saturation, which is the Zaragoza formulation rather than the naive “concatenate the fields and

run plain BM25” version. It also takes an optional entity overlap boost so a candidate sharing a Wikidata entity with a recently clicked article gets credit even when it shares no vocabulary.

`retrieval/lsa.py` is TF-IDF plus truncated SVD at 128 dimensions. `retrieval/sbert.py` is a sentence-transformer encoder backed by a FAISS `IndexFlatIP`. Both expose the same interface so they are swappable by flag. MIND uses `all-MiniLM-L6-v2`. EB-NeRD auto-selects `paraphrase-multilingual-MiniLM-L12-v2` because the corpus is Danish and an English-only encoder on Danish text is a correctness bug, not a tuning choice.

The user query in both cases is built from the last 20 clicked articles with an exponential recency decay of 0.85 per rank step, applied to BM25 term weights and to the mean pooling weights on the embedding side through a shared curve in `retrieval/text_utils.py`.

`retrieval/fusion.py` min-max normalises each scorer inside that impression’s own candidate list and takes a weighted average with the semantic side at 0.7. The normalisation is per impression on purpose. Raw BM25 sums and cosine similarities are not on comparable scales, and normalising globally would let a long candidate list drag a short one around.

Re-ranker. `pipeline/features_{common,mind,ebnerd}.py` build a candidate level feature table: recency-weighted category and subcategory match against history, embedding similarity to the recency-weighted history centroid, article train CTR and impression count, article freshness, candidate position in the impression, and history length. EB-NeRD adds three more from its own behaviour data: prior click count in the session, average past read time, average past scroll percentage. All of these are computed strictly from clicks that precede the impression timestamp.

`scripts/train_reranker.py` fits a LightGBM `LGBMRanker` over those features plus the three Assignment 1 retrieval scores (bm25, semantic, fusion), grouped by impression.

Evaluation. `scripts/evaluate_ranking.py` and `scripts/evaluate_reranker.py` report AUC, MRR, nDCG@5 and nDCG@10 per impression and then averaged, plus intra-list diversity, novelty and catalogue coverage. Everything comes with bootstrap 95% confidence intervals, and before/after comparisons use a paired bootstrap so the CI is on the difference rather than on two independent means. `scripts/ablation_reranker.py` retrains the model with each feature group removed. `scripts/benchmark_serving.py` measures real process RSS incrementally as each index loads and times the full single-request path one impression at a time.

2. Choices and why

Hand rolled BM25F instead of `rank_bm25` or `Elasticsearch`. `rank_bm25` has no field weighting, which was the single change that mattered most on the lexical side. `Elasticsearch` would have given field weights for free but drags in a service, an ingestion step and a query DSL for a workload that is ultimately one sparse matrix multiply. The sparse-matrix version fits in 300 lines, pickles, and lets me add the entity boost as three lines instead of a plugin.

Exact FAISS rather than an approximate index. `IndexFlatIP` on normalised vectors is exact cosine. At 104K articles an exhaustive scan is not the bottleneck, so trading recall for speed would have bought nothing and added a recall parameter to defend. That calculus changes at scale and I say where in section 4.

LSA first, then sentence-transformers. LSA was there to establish that the semantic path worked at all before spending a gigabyte of dependencies on torch. It also turned out to be a useful control. Entity-fused LSA beat BM25F on AUC by a hair. SBERT beat it by a lot. Knowing both numbers tells you the win came from the encoder and not from the general idea of “add a dense signal”.

k1 and b tuned rather than assumed. Textbook defaults of 1.5 and 0.75 were never validated against this data. A grid search on a fixed-seed 20,000 impression subsample moved MIND to `k1=3.0`, `b=1.0` for +0.0053 AUC. I deliberately did not take the literal grid optimum at `k1=3.5` because it was only +0.0007 higher, which is inside subsample noise. EB-NeRD was tuned separately and landed at `k1=2.2`, `b=0.9`. The two datasets have very different candidate list and history shapes, so transferring the MIND values would have been unjustified.

Indices fitted once and persisted. Early on, three scripts each refit BM25 and LSA from raw text with their own CLI flags. The refit cost was trivial but the drift risk was not. If one script’s default changed and the others did not, all three would still look like they were comparing the same method. `scripts/build_indices.py` now

fits once and writes `config.json` next to the pickles, and everything downstream reads hyperparameters back out of that file.

LightGBM rather than a neural ranker. The feature set is 11 to 13 mostly tabular columns with strong monotone structure. Gradient boosted trees are the right tool for that shape, they train on 200K impressions in minutes on a laptop, and **LGBMRanker** handles the impression grouping natively. A two-tower or NRMS style model would need the raw text back in the loop and a GPU budget the assignment does not have.

Per-impression metric averaging. All accuracy metrics are computed inside one impression and then averaged across impressions, not pooled globally. Pooling would let impressions with long candidate lists dominate. This also matches MIND’s official scorer, which I read directly and which caught a real bug: my MRR only used the first hit’s reciprocal rank, while the official formula averages reciprocal rank over every clicked candidate. Identical for single-click impressions, wrong for multi-click ones. Fixed and verified against a from-scratch reimplementation across 3,000 randomised trials with zero mismatches.

3. Observations

3.1 Recall@K and re-ranking disagree about which retriever is better

MIND, val	recall@50	recall@100	recall@200	reranking AUC
BM25F	1.22%	2.00%	3.09%	0.5583
SBERT	0.66%	1.13%	1.97%	0.5860

BM25F wins full-corpus recall at every K and every slice. SBERT wins candidate re-ranking by a wide margin with non-overlapping CIs. The same inversion shows up on EB-NeRD.

This is not a contradiction. Recall@K asks whether the one exact clicked article lands in the top K out of the entire catalogue. SBERT’s embeddings cluster topically similar articles tightly, and for a needle-in-a-haystack question that clustering crowds out the exact match with near-duplicates. Re-ranking asks whether a small pre-filtered candidate list gets ordered well, and there the semantic understanding pays. The leaderboard scores the second task, so I submitted on the second number. Codabench went 0.5805 with plain BM25, then 0.5872 with BM25F plus tuning plus SBERT, then 0.5953 with the entity boost and fusion.

Fusion beat both of its inputs on MIND (AUC 0.5900 against 0.5583 and 0.5860), which is the expected result when two scorers have genuinely different error profiles.

3.2 The re-ranker works on MIND and actively hurts on EB-NeRD

	MIND before	MIND after	EB-NeRD before	EB-NeRD after
AUC	0.5896	0.6343	0.5307	0.5055
MRR	0.2796	0.3145	0.3359	0.3056
nDCG@10	0.3608	0.4040	0.4518	0.4225

Paired bootstrap on MIND gives +0.0448 AUC with a CI of [0.0436, 0.0459]. On EB-NeRD it gives -0.0252 with a CI of [-0.0268, -0.0236]. The regression is real and significant, not noise.

The ablation explains it. Deltas below are full model minus the variant, so positive means the removed feature group was helping.

Variant removed	MIND AUC delta	EB-NeRD AUC delta
All Q1 features (retrieval scores only)	+0.0485	-0.0142
Popularity (train CTR, train impressions)	+0.0131	-0.2027
History content match	+0.0056	+0.0080

Variant removed	MIND AUC delta	EB-NeRD AUC delta
Freshness	+0.0016	+0.0321
Candidate position	+0.0003	-0.0297
Session / engagement	n/a	-0.0059

On EB-NeRD, dropping the popularity features takes AUC from 0.497 to 0.699. Those two columns are not weak, they are poisonous. The cause is in the split boundaries. MIND validates on a single day immediately after training. EB-NeRD validates on a full week. Article popularity measured on train data is a reasonable prior 24 hours out and close to meaningless 7 days out in a news catalogue that turns over daily. The evaluation output confirms this directly: EB-NeRD's 80th percentile of train impression count per validation candidate is **0.0**, meaning most validation candidates were never seen in training at all. The model still leaned on the feature, and it learned a mapping that does not hold in the validation window.

I left the regression in the reported numbers rather than quietly dropping the feature group, because the honest result for EB-NeRD is that this re-ranker configuration is worse than the fusion score it sits on top of. The fix is obvious from the ablation and is stated in section 5.

3.3 The re-ranker trades tail performance for head performance

MIND, sliced by whether the candidate article is in the top 80% by train impression count:

Slice	n	before AUC	after AUC
Head	299,272	0.5864	0.6824
Tail	77,199	0.6018	0.4483

Overall AUC goes up by 0.045 while tail AUC drops by 0.154. The aggregate number hides a model that got substantially worse than random on rarely-seen articles. The beyond-accuracy metrics agree: catalogue coverage falls from 4.47% to 3.11% and novelty falls from 18.17 to 17.30. Intra-list diversity rises from 0.790 to 0.857, which sounds good but mostly reflects the model spreading its picks across categories within a narrower popular set.

Cold-start users go the other way. MIND cold users (fewer than 5 history items) improve from 0.5476 to 0.6196, a +0.072 gain that is larger than the warm-user gain of +0.040. With little history to work from, the popularity and freshness priors are doing exactly what priors should do.

EB-NeRD has zero cold users in validation because its average history length is around 292 items, so that slice is empty rather than uninformative.

3.4 Bugs that changed results

Three worth naming because each one silently produced plausible-looking numbers.

The MIND entity embedding file has a trailing tab, so each line has 102 fields rather than 101. Naming only 101 columns made pandas treat entity id as an index and shift every value over by one. Entity ids came back as floats and 0 of 104,151 articles matched a vector. After the fix, 91,540 matched, which is about 88%. This was caught by a debug script before the downstream results were trusted.

Null handling differed by data path. Parquet returns missing strings as `None`, which is falsy, so `t or ""` worked. `pd.read_csv` returns them as float NaN, which is truthy, so the same guard passed NaN straight into string concatenation. Worked on train and val, crashed on test.

Numpy array truthiness (`if not history_ids`) recurred in three separate files because parquet list columns come back as arrays and `bool()` on a multi-element array raises. Each instance was fixed to an explicit length check.

4. Where it breaks at 10×

The serving benchmark is measured on the real persisted indices, 1,000 sampled impressions, single process, timed one request at a time.

	MIND	EB-NeRD
p50 latency	3.51 ms	29.36 ms
p95	5.90 ms	37.51 ms
p99	11.81 ms	55.71 ms
max	102.86 ms	91.23 ms
Safe QPS per worker at 100 ms p99	~85	~18

Incremental process RSS as each component loads:

Component	MIND	EB-NeRD
Baseline process	143 MB	144 MB
+ re-ranker model	147 MB	147 MB
+ BM25 and semantic index	854 MB	276 MB
+ lookups and feature store	6,704 MB	4,725 MB
+ session/engagement precompute	n/a	4,842 MB

Memory is the first thing that breaks, and it is not the index. The BM25 matrix plus the SBERT embeddings account for about 707 MB on MIND. The user and article lookup tables and feature store account for 5.85 GB, which is 87% of the footprint. Those tables scale linearly in users times history length. At 10× users that is roughly 58 GB of lookups per process, which does not fit on a single commodity box, and every additional worker process holds its own copy because the current design pickles state per worker rather than sharing it. The practical ceiling is not “can we serve 10× traffic” but “can one machine hold one copy of the feature store”. It cannot.

Latency mostly survives 10× more articles and does not survive 10× more candidates. The serving path is restricted-candidate scoring, so per-request cost is $O(\text{candidates})$, not $O(\text{catalogue})$. Growing the catalogue from 104K to 1M articles barely moves p99 on this path. What does move it is candidate list length, and EB-NeRD already shows the shape of that problem. It has one fifth the articles and one third the candidates per impression yet a p99 five times worse, because its history averages 292 items and every feature that touches history scales with that. p99 at 55.7 ms is already over half a 100 ms budget with no headroom for a GC pause or a cache miss.

The full-corpus retrieval path does break. IndexFlatIP is an exhaustive scan, so it is linear in catalogue size by construction. At 1M articles that is 10× the dot products per query, and `evaluate_retrieval.py` already takes about 20 minutes on MIND. That becomes hours. This is the point where the exact-index choice from section 2 stops being free and an IVF or HNSW index with a measured recall tradeoff becomes necessary. The same applies to the BM25 sparse matmul, though it degrades more gracefully because it only touches matched terms.

Offline training breaks on wall clock before it breaks on memory. Building the MIND feature table takes 2 to 2.5 hours over 83.5M candidate rows. At 10× that is a 20 to 25 hour single pass with no checkpointing and no incremental update path. A crash at hour 18 restarts from zero. `ablation_reranker.py` already has to subsample to 200,000 impressions to train six model variants in reasonable time, which means the ablation numbers in section 3.2 come from a smaller training set than the headline model. That gap widens at 10×.

The statistical problem gets worse, not better. Section 3.2 showed EB-NeRD’s popularity features failing because most validation candidates had zero training impressions. More articles per unit time means fewer impressions per article, which means that zero-count problem spreads rather than resolves. Any feature derived

from per-article aggregate counts degrades as the catalogue grows unless the training window grows with it, and in news the training window cannot grow much because the underlying distribution shifts daily.

Cost. At the measured 85 QPS per MIND worker, 10,000 QPS needs 119 workers. At 6.7 GB resident each that is roughly 800 GB of RAM before accounting for the OS. EB-NeRD at 18 QPS per worker needs 558 workers for the same traffic. Neither is a sane deployment. The fix is architectural rather than incremental: move the feature store out of process into a shared key-value store, serve lookups over the network, and accept the added latency in exchange for worker processes that cost hundreds of megabytes rather than gigabytes.

5. What I would do next

Drop the popularity features for EB-NeRD, or recompute them on a rolling window that ends at the impression timestamp instead of at the train boundary. The ablation says this alone moves EB-NeRD AUC from 0.497 to 0.699, which would turn a regression into a large win.

Add a tail-aware objective or a popularity-debiased sample weighting to the MIND re-ranker. A 0.154 AUC drop on the tail slice is too large to accept for a 0.045 overall gain, and coverage falling by a quarter is a product problem even if the leaderboard does not measure it.

Move the feature store behind a shared store and re-benchmark. Everything in section 4 points at the same 5.85 GB of per-process lookup tables.

Port the multiprocessing, entity boost and fusion output from `generate_predictions.py` to the EB-NeRD equivalent. The shared retrieval modules already carry the logic, only the script-level wiring is MIND-only.